

Components

INFO

Lesinhoud

We leren hoe components (Nederlands: componenten) werken in Vue.js. Components zijn herbruikbare stukjes code die je gebruikt om een applicatie op te bouwen.

Doelstellingen

De student(e) kan:

- Werken met een Command Line Interface (CLI) / Node Package Manager (NPM) voor het installeren, updaten en verwijderen van pakketten en modules binnen een project.
- Een single-page frontend applicatie ontwikkelen met Vue.js.
- Zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.

▼ Inhoud

- [Wat is een component?](#)
- [Waarom components gebruiken?](#)
- [Properties \(Props\)](#)
 - [defineProps](#)
- [Slots](#)
 - [Named Slots](#)
- [Emits](#)

- [Events](#)
- [Custom Events met Emits](#)
- [Samenvatting](#)

Wat is een component?

Een Vue-component is een `.vue` -bestand dat bestaat uit drie delen: `<script>`, `<template>`, en `<style>`. Deze delen bevatten respectievelijk de JavaScript-logica, de HTML-structuur en de CSS-styling van de component. Componenten laten je herbruikbare en modulaire code schrijven, waardoor de applicatie beter georganiseerd en onderhoudbaar blijft.

Waarom components gebruiken?

Components maken het mogelijk om herbruikbare en modulaire code te schrijven. Dit betekent dat je dezelfde component op meerdere plaatsen in de applicatie kan gebruiken zonder de code te dupliceren. Dit bevordert de onderhoudbaarheid en leesbaarheid van de code.

Bijvoorbeeld, stel dat er 3 verschillende pagina's zijn die allemaal een knop nodig hebben met dezelfde stijl.

Je kan dezelfde HTML-code en CSS-code in elke pagina kopiëren en plakken.

```
src/views/PageOne.vue
```

```
<template>
  <h1>Pagina 1</h1>
  <button class="my-button">Klik mij</button>
```

vue

```
</template>

<style scoped>
.my-button {
  background-color: blue;
  color: white;
  padding: 10px;
}
</style>
```

src/views/PageTwo.vue

```
<template>
  <h1>Pagina 2</h1>
  <button class="my-button">Klik mij</button>
</template>

<style scoped>
.my-button {
  background-color: blue;
  color: white;
  padding: 10px;
}
</style>
```

vue

src/views/PageThree.vue

```
<template>
  <h1>Pagina 3</h1>
  <button class="my-button">Klik mij</button>
</template>

<style scoped>
```

vue

```
.my-button {  
  background-color: blue;  
  color: white;  
  padding: 10px;  
}  
</style>
```

In dit geval is dezelfde code drie keer geschreven. Dit is niet ideaal, want als je de stijl van de knop moet wijzigen, moet je dit op drie verschillende plaatsen doen.

Oefening

Je kan de drie pagina's aanmaken zoals hierboven beschreven. Zorg dat je de routes toevoegt in `src/router/index.js` zodat je de pagina's kan bezoeken.

In plaats daarvan kan je een component maken voor de knop en deze component in elke pagina gebruiken:

`src/components/MyButton.vue`

```
<template>  
  <button class="my-button">Klik mij</button>  
</template>  
  
<style scoped>  
  .my-button {  
    background-color: blue;  
    color: white;  
    padding: 10px;  
  }  
</style>
```

vue

`src/views/PageOne.vue`

vue

```
<script setup>
import MyButton from "@/components/MyButton.vue";
</script>

<template>
  <h1>Pagina 1</h1>
  <MyButton />
</template>
```

src/views/PageTwo.vue

vue

```
<script setup>
import MyButton from "@/components/MyButton.vue";
</script>

<template>
  <h1>Pagina 2</h1>
  <MyButton />
</template>
```

src/views/PageThree.vue

vue

```
<script setup>
import MyButton from "@/components/MyButton.vue";
</script>

<template>
  <h1>Pagina 3</h1>
  <MyButton />
</template>
```

@-alias import

Merk op dat je de `MyButton` -component in elke pagina importeert en gebruikt in de template. Je gebruikt de `@` -syntax om te verwijzen naar de `src` -map van het project. Dit is een alias die standaard in Vue-projecten is geconfigureerd. Hierdoor voorkom je dat je relatieve paden zoals `../components/MyButton.vue` moet gebruiken, wat onhandig kan zijn bij diep geneste mappenstructuren.

Dit is terug te vinden in het `vite.config.js` -bestand van het project.

```
resolve: {  
  alias: {  
    '@': path.resolve(__dirname, 'src'),  
  },  
},
```

js

De code voor de knop is nu slechts één keer geschreven. Als je de stijl van de knop moet wijzigen, moet je dit alleen in het `MyButton.vue` -bestand wijzigen.

Oefening

Je kan de `MyButton.vue` -component aanmaken en de drie pagina's refactoren zodat je de component gebruikt in plaats van gedupliceerde code.

Properties (Props)

Properties, of `props`, zijn een manier om data mee te geven aan een component. Dit maakt het mogelijk om een component flexibeler en herbruikbaar te maken.

Stel dat je een `MyButton` -component moet maken waarbij je de tekst op de knop via een prop kan aanpassen:

src/components/MyButton.vue

```
<script setup>
defineProps({
  text: {
    type: String,
    required: true,
  },
});
</script>

<template>
  <button class="my-button">{{ text }}</button>
</template>

<style scoped>
.my-button {
  background-color: blue;
  color: white;
  padding: 10px;
}
</style>
```

vue

`{{ text }}` is de syntax waarmee je de waarde van de `text`-prop in de template kan tonen. Dit werkt ook voor variabelen die je in de `<script setup>`-sectie definieert. Dit noemen we interpolatie.

src/views/PageOne.vue

```
<script setup>
import MyButton from "@components/MyButton.vue";
</script>
<template>
  <h1>Pagina 1</h1>
```

vue

```
<MyButton text="Klik mij op Pagina 1" />
</template>
```

src/views/PageTwo.vue

```
<script setup>
import MyButton from "@components/MyButton.vue";
</script>
<template>
  <h1>Pagina 2</h1>
  <MyButton text="Klik mij op Pagina 2" />
</template>
```

vue

src/views/PageThree.vue

```
<script setup>
import MyButton from "@components/MyButton.vue";
</script>
<template>
  <h1>Pagina 3</h1>
  <MyButton text="Klik mij op Pagina 3" />
</template>
```

vue

De component is nu flexibeler omdat je de tekst op de knop kan wijzigen door een **prop** mee te geven.

Parent/Child

We spreken vaak over een parent-child-relatie tussen componenten. De parent-component is de component die een andere component (child) gebruikt. In het bovenstaande voorbeeld zijn **PageOne.vue**, **PageTwo.vue** en **PageThree.vue** de parent-componenten, terwijl **MyButton.vue** de child-

component is. De parent-componenten geven data (zoals de tekst voor de knop) door aan de child-component via props.

defineProps

In Vue 3 gebruik je de `defineProps` -functie om props te definiëren in een component. `defineProps` accepteert een object als argument, waarin je de naam van de prop en een configuratie voor die prop kan opgeven, zoals het type en of het verplicht is.

```
<script setup>
defineProps({
  text: {
    type: String,
    required: true,
  },
});
</script>
```

vue

In dit voorbeeld is een prop genaamd `text` gedefinieerd met het type `String`. Er is aangegeven dat deze prop verplicht is (`required: true`) om aan de component mee te geven. Je kan types zoals `String`, `Number`, `Boolean`, `Array`, `Object` en `Function` gebruiken. Dit is optioneel, maar het helpt bij het valideren van de props die je aan de component doorgeeft.

CONVENTIE

Binnen dit opleidingsonderdeel is ervoor gekozen om altijd de object-syntax te gebruiken bij `defineProps`. Dit maakt explicieter en duidelijker wat de verwachtingen zijn voor elke prop.

niet doen

```
<script setup>
defineProps(["text"]);
```

vue

```
</script>
```

wel doen

```
<script setup>
defineProps({
  text: {
    type: String,
    required: true,
  },
});
</script>
```

vue

Oefening

Je kan de `MyButton` -component aanpassen zodat deze een prop `text` accepteert om de tekst op de knop te bepalen. Refactor vervolgens de drie pagina's (parent-componenten) zodat je verschillende teksten doorgeeft aan de `MyButton` -component (child-component).

Slots

Een slot is een andere manier om content door te geven aan een component. In plaats van een specifieke prop te gebruiken, kan je een slot gebruiken om aan te geven waar je bepaalde content wilt plaatsen in de component.

Stel dat je de `MyButton` -component zodanig aanpast dat je de inhoud van de knop via een slot plaatst (het `default slot` genoemd):

`src/components/MyButton.vue`

```
<template>
  <button class="my-button">
```

vue

```

    <slot></slot>
  </button>
</template>

<style scoped>
.my-button {
  background-color: blue;
  color: white;
  padding: 10px;
}
</style>

```

Dan kan je de component zoals hieronder gebruiken:

src/views/PageOne.vue

```

<script setup>
import MyButton from "@components/MyButton.vue";
</script>

<template>
  <h1>Pagina 1</h1>
  <MyButton>Klik mij op Pagina 1</MyButton>
</template>

```

vue

Merk op dat je de tekst voor de knop nu tussen de begin- en eindtag van de **MyButton**-component plaatst. Dit is de inhoud die je in het slot van de component invoegt.

Oefening

Je kan de **MyButton**-component aanpassen zodat deze een slot gebruikt om de inhoud van de knop te bepalen. Refactor vervolgens de drie pagina's (parent-componenten) zodat je verschillende teksten via het slot invoegt.

Named Slots

Named slots maken het mogelijk om meerdere slots in een component te definiëren, elk met een unieke naam. Dit is handig wanneer je verschillende delen van een component wilt invullen met verschillende inhoud.

Stel dat je een `MyCard` -component maakt met een header, body en footer:

`src/components/MyCard.vue`

```
<template>
  <div class="card">
    <header class="card-header">
      <slot name="header"></slot>
    </header>
    <div class="card-body">
      <slot name="body"></slot>
    </div>
    <footer class="card-footer">
      <slot name="footer"></slot>
    </footer>
  </div>
</template>

<style scoped>
.card {
  border: 1px solid #ccc;
  border-radius: 5px;
  padding: 10px;
  display: flex;
  flex-direction: column;
}
.card-header,
.card-footer {
  background-color: #f5f5f5;
  color: #333;
```

vue

```
padding: 5px;
}
.card-body {
padding: 10px;
flex: 1;
}
</style>
```

Dan kan je de component zoals hieronder gebruiken:

src/views/PageOne.vue

```
<script setup>
import MyCard from "@components/MyCard.vue";
import MyButton from "@components/MyButton.vue";
</script>

<template>
  <h1>Pagina 1</h1>
  <MyCard>
    <template #header>
      <h2>Card Header</h2>
    </template>
    <template #body>
      <p>Dit is de body van de kaart.</p>
    </template>
    <template #footer>
      <MyButton>Klik mij</MyButton>
    </template>
  </MyCard>
</template>
```

vue

Merk op dat je alle html-elementen en eigen componenten tussen de begin- en eindtags van een named slot kan plaatsen. **MyButton** is een component die je in de footer van de **MyCard** -component plaatst.

Oefening

Refactor de `MyCard` -component zodat je een `default slot` gebruikt in plaats van een `named slot` voor de card-body. Refactor vervolgens het gebruik van de `MyCard` -component in `PageOne.vue` om de inhoud via het slot door te geven.

▼ Oplossing

`src/components/MyCard.vue`

```
<template>
  <div class="card">
    <header class="card-header">
      <slot name="header"></slot>
    </header>
    <div class="card-body">
      <slot></slot>
    </div>
    <footer class="card-footer">
      <slot name="footer"></slot>
    </footer>
  </div>
</template>

<style scoped>
.card {
  border: 1px solid #ccc;
  border-radius: 5px;
  padding: 10px;
  display: flex;
  flex-direction: column;
}
.card-header,
.card-footer {
  background-color: #f5f5f5;
  color: #333;
  padding: 5px;
}
.card-body {
  padding: 10px;
  flex: 1;
}
```

vue

```
}  
</style>
```

src/views/PageOne.vue

```
<script setup>  
import MyCard from "@components/MyCard.vue";  
import MyButton from "@components/MyButton.vue";  
</script>  
  
<template>  
  <h1>Pagina 1</h1>  
  <MyCard>  
    <template #header>  
      <h2>Card Header</h2>  
    </template>  
    <!-- <template #body> -->  
    <p>Dit is de body van de kaart.</p>  
    <!-- </template> -->  
    <template #footer>  
      <MyButton>Klik mij</MyButton>  
    </template>  
  </MyCard>  
</template>
```

vue

Emits

Via emits kan je custom events versturen van een child-component naar een parent-component. Dit is handig wanneer een child-component een actie uitvoert die de parent-component moet weten, zoals het klikken op een knop of het invullen van een formulier. Voordat we naar custom events met emits kijken, bekijken we eerst hoe je standaard DOM-events in een component kan afhandelen.

Events

In Vue.js kan je standaard DOM-events afhandelen met behulp van de `@`-syntax in de template. Dit is een verkorte manier om een event listener aan een HTML-element (of custom component) toe te voegen.

Stel dat je wilt vaststellen of er op de `MyButton`-component wordt geklikt. Je doet dit door een `click`-event listener aan de component in de parent-component toe te voegen:

`src/views/PageOne.vue`

```
vue
<script setup>
import MyButton from "@components/MyButton.vue";

const handleClick = () => {
  alert("De knop is geklikt!");
};
</script>

<template>
  <h1>Pagina 1</h1>
  <MyButton @click="handleClick">Klik mij op Pagina 1</MyButton>
</template>
```

In dit voorbeeld is in de parent-component (`PageOne.vue`) een `handleClick`-functie gedefinieerd. Vervolgens is een `@click`-event listener aan de `MyButton`-component toegevoegd, die de `handleClick`-functie aanroept wanneer je op de knop klikt.

Custom Events met Emits

Wanneer je wilt vaststellen of er ergens op de `MyCard`-component wordt geklikt, gebruik je een normaal `@click`-event zoals hierboven beschreven.

Wanneer je specifiek wilt vaststellen of er op de header, body of footer van de `MyCard` -component is geklikt, kan je custom events met `emits` toepassen.

`src/components/MyCard.vue`

```
vue
<script setup>
  const emit = defineEmits(["headerClick", "bodyClick",
    "footerClick"]);
</script>

<template>
  <div class="card">
    <header class="card-header"
      @click="emit('headerClick')">
      <slot name="header"></slot>
    </header>
    <div class="card-body" @click="emit('bodyClick')">
      <slot></slot>
    </div>
    <footer class="card-footer"
      @click="emit('footerClick')">
      <slot name="footer"></slot>
    </footer>
  </div>
</template>
```

Via `defineEmits` definieer je de mogelijke custom events die deze component kan versturen. In dit geval zijn dat `headerClick`, `bodyClick` en `footerClick`. Vervolgens gebruik je de `emit` -functie om deze events te versturen wanneer je op de respectievelijke delen van de kaart klikt.

Deze events kan je vervolgens in de parent-component afhandelen:

`src/views/PageOne.vue`

```
<script setup>
import MyCard from "@components/MyCard.vue";
const handleHeaderClick = () => {
  alert("De header is geklikt!");
};
const handleBodyClick = () => {
  alert("De body is geklikt!");
};
const handleFooterClick = () => {
  alert("De footer is geklikt!");
};
</script>

<template>
  <h1>Pagina 1</h1>
  <MyCard
    @headerClick="handleHeaderClick"
    @bodyClick="handleBodyClick"
    @footerClick="handleFooterClick"
  >
    <template #header>
      <h2>Card Header</h2>
    </template>
    <p>Dit is de body van de kaart.</p>
    <template #footer>
      <MyButton>Klik mij</MyButton>
    </template>
  </MyCard>
</template>
```

In dit voorbeeld definieer je drie functies in de parent-component (`PageOne.vue`) om de custom events af te handelen. Vervolgens voeg je `@headerClick` , `@bodyClick` en `@footerClick` event listeners toe aan de `MyCard` -component, die de respectieve functies aanroepen wanneer de component de custom events verstuurt. De event listeners bestaan uit een `@` gevolgd door de naam van het event dat wordt geëmit.

Oefening

Je kan de `MyCard` -component aanpassen zodat deze custom events `headerClick` , `bodyClick` en `footerClick` emit wanneer je op de respectievelijke delen van de kaart klikt. Refactor vervolgens de `PageOne.vue` -component zodat je deze custom events afhandelt en een alert toont wanneer je op elk deel klikt.

Samenvatting

Wanneer je herhalende code in een applicatie aantreft, heeft het de voorkeur om deze code naar een component te verplaatsen. Via `props` geef je data door van een parent-component naar een child-component. Via `slots` geef je content door van een parent-component naar een child-component. Via `emits` stuur je custom events van een child-component naar een parent-component.

In andere frameworks noemen ze `props` ook wel `inputs` en `emits` ook wel `outputs` .

Wanneer je een component aanmaakt, denk je na over welke data en functionaliteit de component nodig heeft om goed te functioneren. Dit helpt bij het definiëren van de juiste `props` en `emits` voor de component, waardoor deze flexibeler en herbruikbaar wordt.

Previous page
Oefeningen

Next page
Oefeningen

